

Sistemas en Tiempo Real

3º Curso – Grado en Ingeniería Informática
Especialidad Ingeniería de Computadores



Sistemas en Tiempo Real

Programa de Teoría

Tema 1: Introducción a los Sistemas en Tiempo Real (2 h.)

Tema 2: Lenguajes para aplicaciones en Tiempo Real (2 h.)

Tema 3: Interfaces y Elementos Hardware (4 h.)

Tema 4: Concurrencia y Sincronización (6 h.)

Tema 5: Sistemas Operativos en Tiempo Real (6 h.)

Tema 6: Planificación de Tiempo Real (6 h.)



Tema 6. Planificación de Tiempo Real



1. Estrategias de planificación

1. Definición y necesidad de planificación en un S.O.T.R.
2. Planificación cíclica vs. Planificación apropiativa

2. Estructuración de prioridades

3. Asignación de prioridades

4. Interacción entre Procesos y Bloqueos



Tema 6. Planificación de Tiempo Real

- Definición y necesidad de planificación en un SOTR.
 - En el momento en que hay más de 2 tareas que se ejecutan en un determinado sistema hay que plantear algún mecanismo para poder intercambiar las tareas.
 - El **planificador** es un módulo del S.O. que se encarga de controlar las tareas existentes en el sistema y proporcionarles tiempo de CPU en un determinado instante, gestionando el estado de ejecución en el que se encuentra y decidiendo en cada caso qué tarea debe ejecutarse en cada momento.
 - La planificación, en un SO de propósito general (SOPG), intenta garantizar un uso extensivo de la CPU, equidad en la distribución de acceso al tiempo de ejecución y bajo *overhead*.
 - Se dice que la planificación de un conjunto de tareas es **viable** si todas las tareas comienzan su ejecución después de su instante de lanzamiento y todas terminan antes de su *deadline*.

Tema 6. Planificación de Tiempo Real

• Planificación cíclica vs. Planificación apropiativa

- La planificación de las tareas tiene 2 mecanismos principales:

- Planificación **cíclica**:

- Las tareas se ejecutan repetitivamente con un determinado periodo. El S.O. permite ejecutar cada tarea sin interrupción por parte de las otras tareas hasta que finalice, tras lo cual se ejecuta otra tarea.

- Planificación **apropiativa**:

- El S.O. asigna la CPU a una tarea hasta que la interrumpe y le proporciona la CPU a otra tarea distinta. Se dice que es apropiativa (o expropiativa) porque el S.O. se apropia de la CPU, que estaba en posesión de una tarea. La asignación de una tarea a una CPU puede realizarse por un cierto tiempo conocido a priori o hasta que suceda un bloqueo o llamada al S.O.

Tema 6. Planificación de Tiempo Real

Planificación cíclica

- La planificación cíclica es un mecanismo de planificación:
 - Sencillo de implementar.
 - Calculable a priori.
 - Muy eficiente en términos de bajo *overhead*.
 - No maneja bien eventos esporádicos ni interrupciones.
 - Requiere un conocimiento muy concreto de los tiempos de ejecución de las rutinas.
 - Muy restrictivo ya que todas las tareas deben tener tiempos de ejecución similares.
- Se utiliza principalmente en pequeños sistemas empotrados donde se conocen completamente las tareas que requiere el sistema.
- Para desarrollar un planificador basta con diseñar un bucle que vaya ejecutando una tras otra las tareas.
- Hay que definir un ciclo mayor, coincide con el periodo de repetición de toda la planificación (y es el mínimo común múltiplo de todos los periodos) y un ciclo menor, que es el máximo común divisor de todas las tareas.
- Al menos una vez en cada ciclo mayor habrá que ejecutar la tarea de mayor periodo y al menos una vez cada ciclo menor, habrá que ejecutar la tarea de menor periodo.



Tema 6. Planificación de Tiempo Real

• Ejemplo de Planificación cíclica (sin prioridad)

Proceso	Periodo	Tiempo Ejecución
A	25	10
B	25	8
C	50	5
D	50	4
E	100	2

```

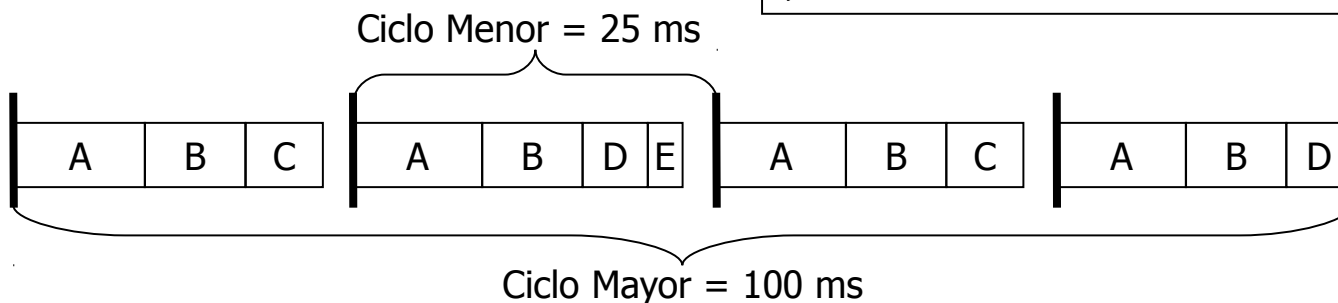
while(true) { // Ciclo Mayor = 100 ms.
    // Ciclo Menor: interrupción cada 25 ms.

    wait(interrupt);
    A(); B(); C();           // Ciclo Menor (23 ms)

    wait(interrupt);
    A(); B(); D(); E();     // Ciclo Menor (24 ms)

    wait(interrupt);
    A(); B(); C();         // Ciclo Menor (23 ms)

    wait(interrupt);
    A(); B(); D();         // Ciclo Menor (22 ms)
}
    
```



Tema 6. Planificación de Tiempo Real

• Planificación apropiativa

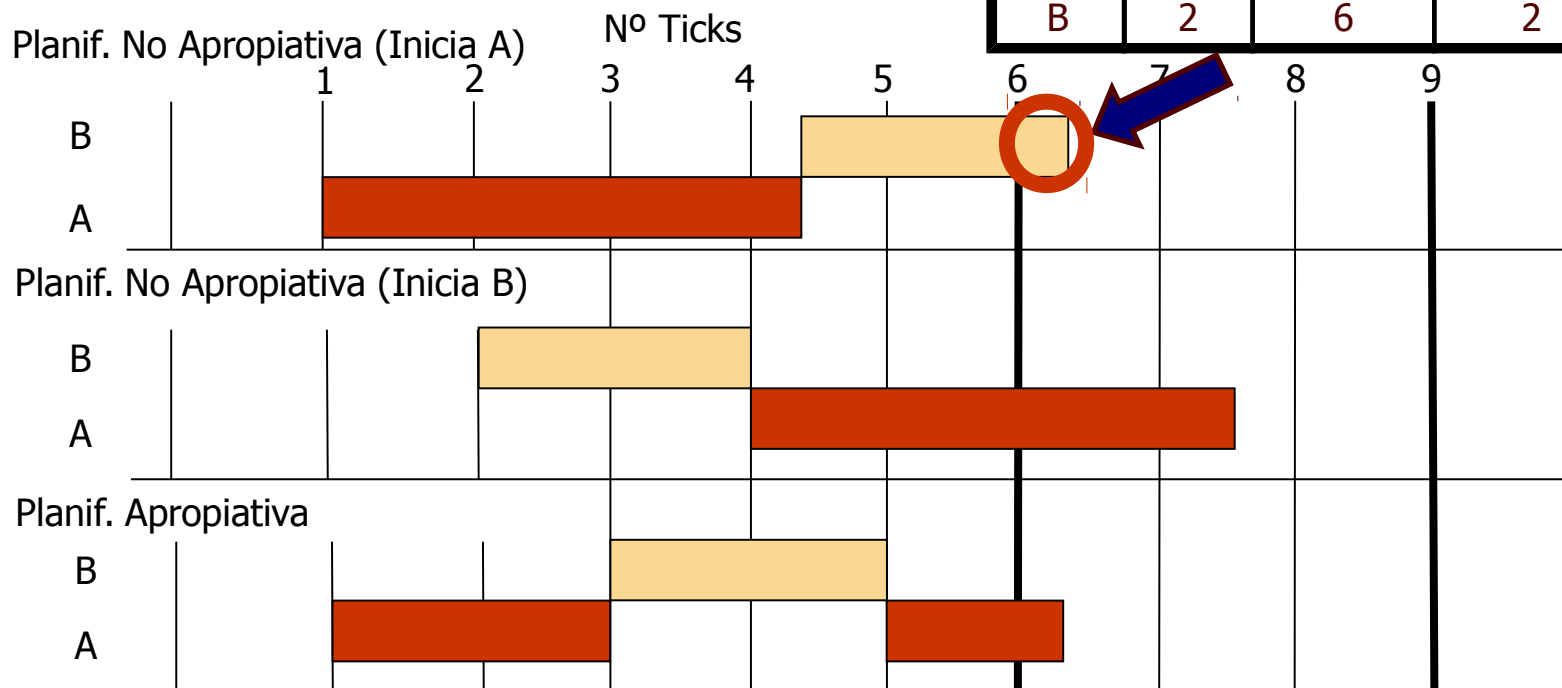
- La planificación apropiativa es un mecanismo de planificación:
 - Flexible.
 - Permite manejar de manera eficaz los eventos esporádicos y las interrupciones.
 - Permite reducir los tiempos ociosos de la CPU.
 - Elevada complejidad en el S.O.
 - Elevado *overhead* del S.O.
- El método más habitual es el de *round-robin* (también llamado *time-slicing*):
 - A cada tarea se le asigna un tiempo fijo de CPU y se le permite la ejecución hasta que agota dicho tiempo o hasta que realice una operación que bloquee la tarea.
 - En el método de planificación *round-robin* tradicional todas las tareas tienen asignada la misma prioridad.



Tema 6. Planificación de Tiempo Real

Planificación apropiativa vs. No apropiativa

Tarea	Inicio	Deadline	Ejecución
A	1	9	3.25
B	2	6	2



Tema 6. Planificación de Tiempo Real

1. Estrategias de planificación



2. Estructuración de prioridades

1. Definición y necesidad de prioridades en un S.O. en Tiempo Real
2. Niveles de prioridad: nivel de interrupción, de reloj y de prioridad base
3. Problemas de prioridad en tareas repetitivas: "jitter", tareas con ejecuciones mayores que un *tick*

3. Asignación de prioridades

4. Interacción entre Procesos y Bloqueos

Tema 6. Planificación de Tiempo Real

• Estructuración de prioridades

- En los SOTR no todas las tareas tienen el mismo nivel de importancia, por tanto, hay que crear un mecanismo para asignar y manejar prioridades de las tareas.
- Usualmente, las prioridades se asignan mediante un valor entero no negativo, aunque esto depende de cada S.O.
- Frente a los SOPG, los SOTR no pretenden garantizar la equidad en el acceso a la CPU, sino que TODAS las tareas de mayor prioridad cumplen sus *deadlines*, aunque ello signifique que las tareas de baja prioridad no se ejecutan nunca.
- Determinar la prioridad de los procesos es una tarea muy compleja que involucra tanto al SOTR como al programador del sistema y al administrador del mismo.



Tema 6. Planificación de Tiempo Real

• Estructuración de prioridades

- Las prioridades se pueden asignar de dos modos:
 - Prioridades fijas o estáticas:
 - Cada tarea tiene asignada una prioridad que se mantiene mientras la aplicación está en ejecución.
 - Prioridades dinámicas:
 - Cada tarea tiene una prioridad que puede variar durante la ejecución del sistema.
- Ventajas de las prioridades dinámicas:
 - Los esquemas de prioridad dinámica aumentan la flexibilidad (permiten aumentar la prioridad de una tarea en una situación de alerta).
- Desventajas de las prioridades dinámicas:
 - El cambio dinámico de prioridades es peligroso porque el comportamiento del sistema es mucho más complejo de predecir.
 - También existe el riesgo de bloquear algunas tareas durante demasiado tiempo.



Tema 6. Planificación de Tiempo Real

• Niveles de prioridad

- Con carácter general, las prioridades se estructuran en tres niveles principales:
 - **Nivel de Interrupción:** Máximo nivel de prioridad. Este nivel es el utilizado para RTI de tareas y dispositivos que requieren una respuesta muy rápida (del orden de milisegundos). Dentro de este nivel se ejecuta la tarea de control de tiempo real o del despachador.
 - **Nivel de Reloj:** Este nivel está asociado con la rutina de reloj de tiempo real y es el utilizado para tareas cíclicas y repetitivas que tengan alta precisión. Dentro de este nivel se ejecuta el planificador.
 - **Nivel de Prioridad Base:** Es el nivel para las tareas de baja prioridad, que no tienen un *deadline* estricto o que tienen un margen de error amplio en sus temporizaciones.
- Cada nivel de prioridad suele tener su lista de tareas independiente y diferentes mecanismos de planificación.



Tema 6. Planificación de Tiempo Real

• Nivel de interrupción

- Las interrupciones realizan generalmente tareas asíncronas que fuerzan un cambio de planificación de lo que esté ejecutando la CPU, que son difíciles de controlar y medir temporalmente.
- Las RTI deben ocupar el mínimo tiempo posible de CPU. Generalmente, realizan pequeñas modificaciones de variables compartidas y generan señales a rutinas con niveles de prioridad inferior para que realicen determinadas tareas más complejas.
- Dentro del nivel de interrupción habrá diversos niveles de manera que existan interrupciones más prioritarias que otras.



Tema 6. Planificación de Tiempo Real

• Nivel de reloj

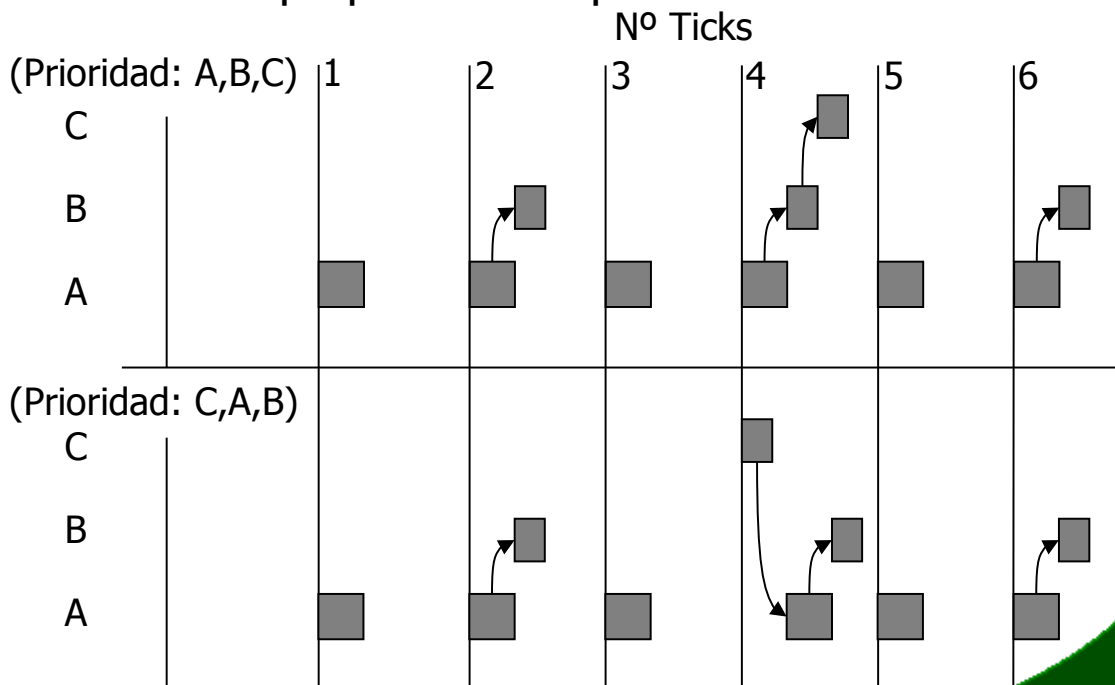
- Dentro de las rutinas con prioridad de nivel de interrupción es la que controla el reloj de tiempo real.
- El reloj de tiempo real se ejecuta repetitivamente con un periodo de entre 1 y 200 ms.
- Cada interrupción del reloj de tiempo real se denomina *tick*, representa el periodo de tiempo menor que es capaz de tratar el sistema de manera fiable.
- La rutina del reloj de tiempo real tiene como objetivo mantener el reloj-calendario del sistema y transferir el control al despachador. El planificador es el encargado de seleccionar la tarea que debe ejecutarse en cada *tick*.
- Las tareas que se encuentran en el nivel de prioridad de reloj se pueden englobar en 2 tipos:
 - **Cíclicas:** Son Tareas periódicas que requieren una sincronización muy precisa.
 - **Retardadas:** Son tareas que requieren tener un retardo fijo entre sucesivas repeticiones o simplemente que quieren retrasar su actividad durante un periodo de tiempo.



Tema 6. Planificación de Tiempo Real

- Nivel de reloj: Prioridad de las tareas cíclicas
 - Las tareas cíclicas suelen tener una prioridad acorde con la precisión de la temporización requerida, es decir, suelen tener mayor prioridad cuanto más pequeño es el periodo.

Tarea	Precisión (Ticks)
A	1
B	2
C	4



Tema 6. Planificación de Tiempo Real

● Nivel de reloj: Rutinas retardadas

- Las tareas pueden pedirle al SOTR un retardo preciso en su ejecución, de manera que la tarea pasa de "ejecutándose" a "bloqueada" (o "suspendida") durante dicho periodo de tiempo.
- Para controlar estas peticiones, en el nivel de prioridad de reloj se encuentran los gestores de temporización, que se encargan de programar alarmas para dormir o despertar a una determinada tarea tras un cierto número de *ticks*.
- Muchos SOTR no soportan tareas cíclicas y debe ser el usuario el que cree temporizaciones precisas (utilizando los gestores de temporización) que planifiquen una nueva ejecución de la tarea con un cierto retardo, para conseguir que se ejecuten cíclicamente.

Tema 6. Planificación de Tiempo Real

- **Implementación de control de Rutinas retardadas**
 - Para implementar el retardo se suele usar un cola (o una lista ordenada) donde se colocan las tareas que han solicitado retardo, ordenadas por el tiempo de finalización más próximo.
 - Cuando una tarea solicita un retardo, llama al gestor de temporización, que calcula el tiempo y la introduce ordenadamente en la lista.
 - Una tarea que se ejecuta en el nivel de prioridad de reloj, chequeará la cola y comprobará si el tiempo asociado a la primera tarea de la cola ha cumplido su tiempo. Si lo ha cumplido, se retira de la cola y dicha tarea se pasa a la cola de tareas "preparadas" para su posible ejecución.
 - Se continúa comprobando la lista hasta que se encuentre una tarea que ya no cumpla su tiempo.
 - La tarea que realiza el chequeo suele ser el despachador de tareas que se ejecuta en cada interrupción de reloj. Si se asocia a otra tarea que se ejecute con menor frecuencia, estaríamos trabajando a un nivel de prioridad equivalente a la prioridad base del planificador.



Tema 6. Planificación de Tiempo Real

• Nivel de prioridad base

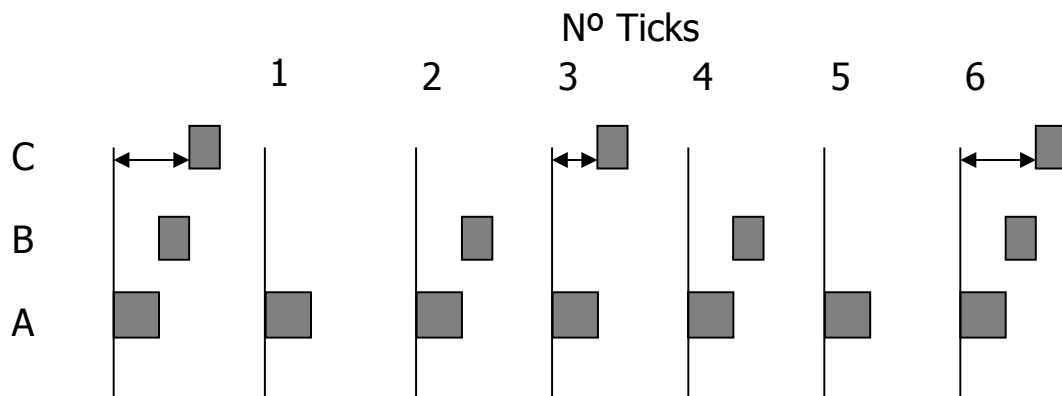
- Este nivel de prioridad está asociado a las tareas de menor nivel de prioridad e importancia en un SOTR.
- Las tareas de nivel de prioridad base suelen tener su ejecución asociado a peticiones más que por intervalos determinados.
- Estas peticiones pueden ser eventos externos de dispositivos lentos, o bien, peticiones unidas a requerimientos particulares sobre los datos que se tienen que procesar.
- En los SOTR no se suele utilizar el *round-robin* básico para planificar estas tareas (aunque sí en otros niveles de prioridad) ya que siempre suele haber otras tareas prioritarias pendientes de ejecución y que hay que tener en cuenta.
- En los SOTR se utilizan estrategias de prioridades dentro de las tareas del nivel de prioridad base. Se utilizan tanto asignación estática de prioridades (para tareas dentro de una jerarquía clara de prioridades) como dinámica (para tareas más flexibles y adaptables a las circunstancias).



Tema 6. Planificación de Tiempo Real

- Problemas de prioridad en tareas repetitivas: "jitter"
 - El *jitter* es la variación dentro de su intervalo entre diversas ejecuciones periódicas de una tarea de baja prioridad, debido a que tiene que esperar a que finalicen el resto de tareas de mayor prioridad.

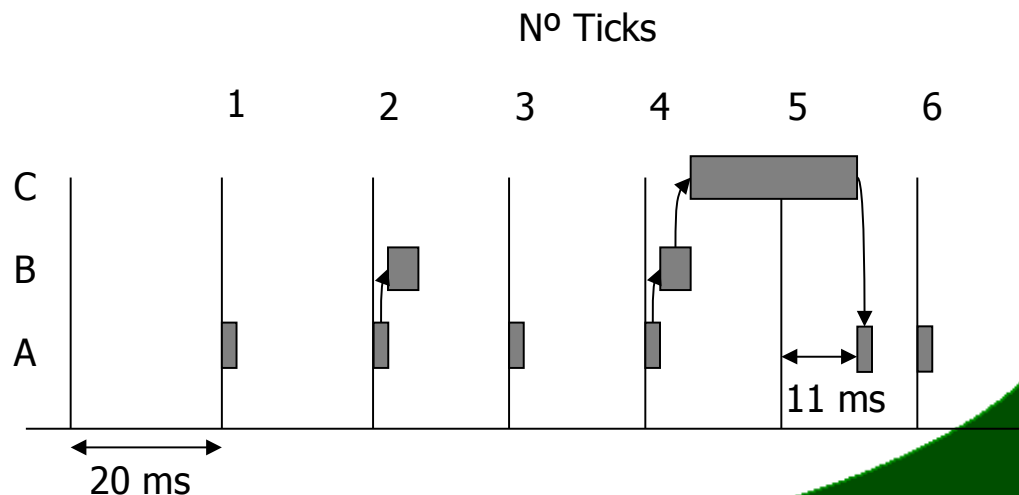
Tarea	Precisión (Ticks)
A	1
B	2
C	3



Tema 6. Planificación de Tiempo Real

- Problemas de prioridad en tareas repetitivas: tareas con ejecuciones mayores de un *tick*
 - Si son interrumpibles no hay problema, si no son interrumpibles, estas tareas producirán desfases cíclicos en otras tareas (que incluso pueden ser de mayor prioridad).
 - Producen "jitter" en tareas de menor prioridad o mayor precisión.
 - Para un correcto funcionamiento del sistema no deben ejecutarse con una frecuencia elevada.

Tarea	Tiempo (ms.)	Precisión (Ticks)
A	1	1
B	6	2
C	25	4



Tema 6. Planificación de Tiempo Real

1. Estrategias de planificación
2. Estructuración de prioridades
- ➔ 3. Asignación de prioridades
 1. Conceptos sobre prioridades y estrategias apropiativas
 2. Prioridad Estática: Algoritmo de la Razón Monótona (RMA)
 3. Test de planificabilidad utilizando RMA
 4. Algoritmo de Ordenación de Prioridad por "Deadline" Monótono (DMPO)
 5. Prioridad Dinámica: Deadline más cercano (EDF)
3. Interacción entre Procesos y Bloqueos



Tema 6. Planificación de Tiempo Real

Asignación de prioridades

- La asignación de prioridades a las tareas se puede realizar con planificaciones apropiativas y con no apropiativas. Lo más habitual es que se realice con planificaciones apropiativas.
- Las prioridades pueden asignarse de manera estática o de manera dinámica.
- Suposiciones habituales en la asignación de prioridades para planificaciones apropiativas:
 - **S1:** El cambio de contexto es despreciable frente a la carga computacional de las tareas. Además, el cambio de contexto se puede realizar en cualquier momento (el tamaño de las secciones críticas es minúsculo). El número de veces que una tarea es interrumpida no tiene influencia en el rendimiento del algoritmo de planificación.
 - **S2:** Las tareas solo tienen requisitos importantes de tiempo de ejecución y no consumen tiempo (o es despreciable) en llamadas al sistema para pedir recursos.
 - **S3:** Las tareas tienen un comportamiento independiente (ninguna o muy pocas Secciones Críticas).
- Estas suposiciones permiten simplificar el análisis de los algoritmos de planificación.



Tema 6. Planificación de Tiempo Real

- **Asignación estática de Prioridad: Razón Monótona**
 - El Algoritmo de la Razón Monótona (RMA) es uno de los algoritmos de asignación de prioridad más estudiados y es el más utilizado en la práctica.
 - Además de las 3 suposiciones anteriores, para RMA se toman 2 suposiciones más:
 - **S4:** Todas las tareas son periódicas.
 - **S5:** El *deadline* (relativo, es decir, en cada iteración de la tarea) de cada tarea es igual a su periodo.
 - Este algoritmo asigna prioridades (P_i) a las tareas inversamente proporcionales a su periodo (T_i). Si $T_i < T_j$ entonces $P_i > P_j$.
 - Las tareas de mayor prioridad pueden interrumpir a las tareas de menor prioridad.
 - El esquema de prioridades de RMA es óptimo, en tanto que si un conjunto de tareas es planificable mediante cualquier mecanismo de prioridades fijas, entonces lo será mediante RMA.

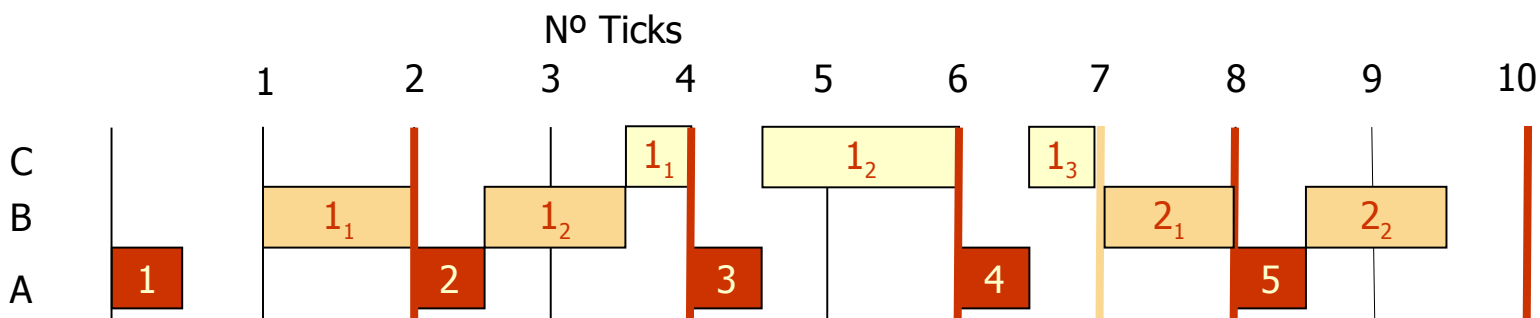


Tema 6. Planificación de Tiempo Real

- Asignación de prioridades estáticas mediante RMA

Tarea	Inicio	Periodo	Ejecución	Prioridad
A	0	2	0.5	50
B	1	6	2	17
C	3	10	2.5	10

$$\text{Prioridad}_i = \frac{100}{P_i}$$

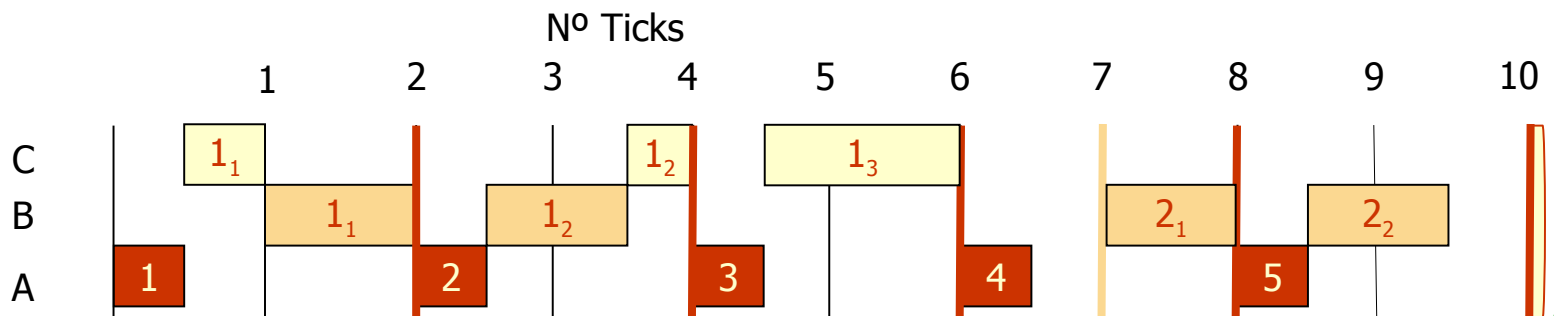


Tema 6. Planificación de Tiempo Real

- Asignación de prioridades estáticas mediante RMA

Tarea	Inicio	Periodo	Ejecución	Prioridad
A	0	2	0.5	50
B	1	6	2	17
C	0	10	2.5	10

$$\text{Prioridad}_i = \frac{100}{P_i}$$

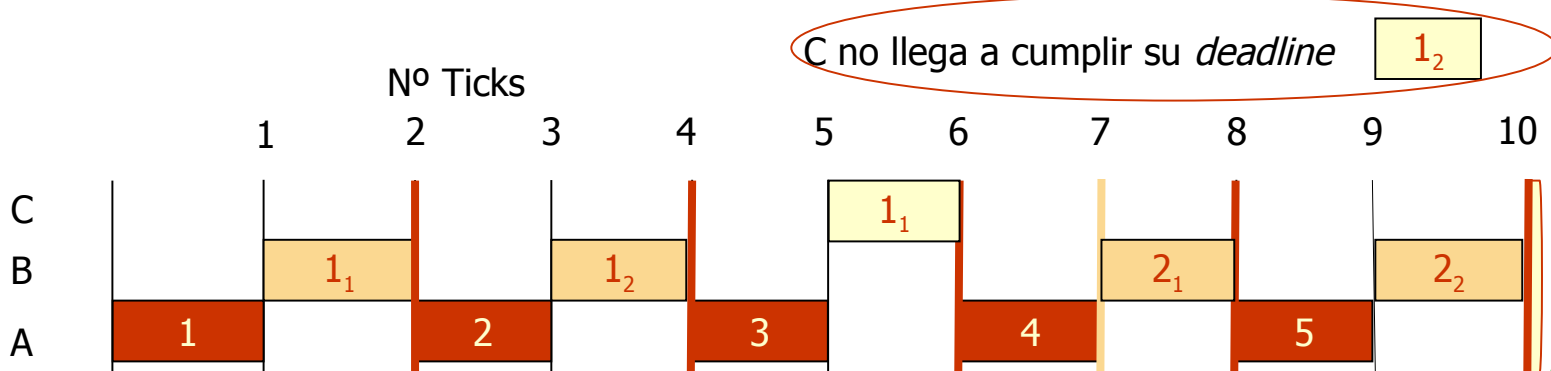


Tema 6. Planificación de Tiempo Real

- Asignación de prioridades estáticas mediante RMA

Tarea	Inicio	Periodo	Ejecución	Prioridad
A	0	2	1	50
B	1	6	2	17
C	0	10	1.75	10

$$\text{Prioridad}_i = \frac{100}{P_i}$$



Tema 6. Planificación de Tiempo Real

• Test de planificabilidad utilizando RMA

- El algoritmo de la Razón Monótona posee un mecanismo matemático que permite determinar si un conjunto de tareas es viable utilizando RMA.
- En 1973, Liu y Layland demostraron que un conjunto de tareas es planificable utilizando RMA si:

- Para N procesos, se cumple que:

$$\sum_{i=1}^N \left(\frac{C_i}{T_i} \right) < N \cdot \left(2^{\frac{1}{N}} - 1 \right)$$

- Donde C_i es el tiempo de ejecución en el peor caso del proceso i.
- T_i es el periodo de repetición del proceso (llamado periodo de lanzamiento).
- La sumatoria representa el grado de utilización total de la CPU por el conjunto de procesos.



Tema 6. Planificación de Tiempo Real

• Test de planificabilidad utilizando RMA

- En la tabla se muestra el grado de utilización de la CPU para distintos valores de N.
- Para valores elevados de N, el límite se aproxima asintóticamente a 69.3%. Es decir, si para cualquier conjunto de procesos se consigue que su utilización de la CPU sea menor del 69.3% del tiempo, los procesos serán siempre planificables utilizando RMA.
- Este test es solo una condición suficiente y no necesaria. Si se cumple el test, entonces se cumplirán todos los *deadline*. Si el test falla, los *deadline* podrían cumplirse o no.
- Si el test falla, deberíamos hacer el gráfico de *timeline* para comprobar el comportamiento de las tareas y observar si son planificables o no.

N	Grado de Utilización (%)
1	100.0 %
2	82.8 %
3	78.0 %
4	75.7 %
5	74.3 %
10	71.8 %
∞	69.3 %



Tema 6. Planificación de Tiempo Real

- Test de planificabilidad utilizando RMA
 - Mostremos un ejemplo de utilización.

	Periodo (T)	T. Comput. (C)	Prioridad (P)	Utilización (U = C/T)
T1	50	12	1	0.24
T2	40	10	2	0.25
T3	30	10	3	0.33
Utilización Global de la CPU				0.82

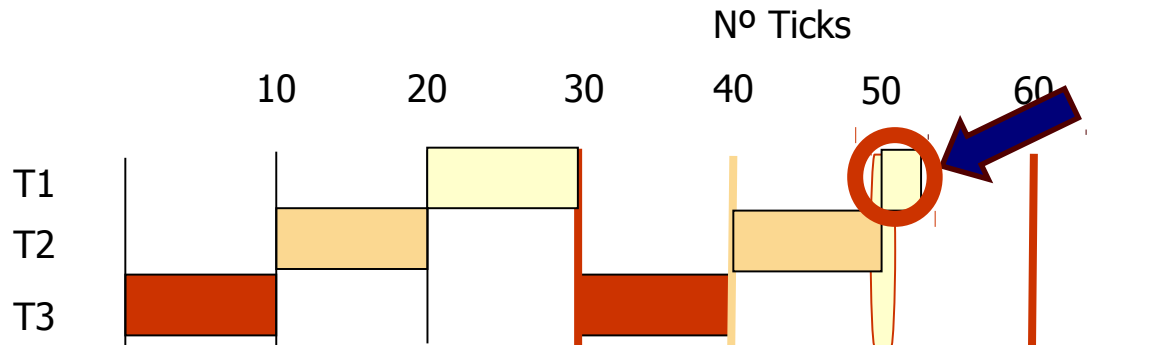
- La utilización de la CPU es de 0.82, mientras que $N(2^{1/N}-1)$ es igual a $3(2^{1/3}-1) = 0.78$; por lo que no se cumple el test de planificabilidad.
- Mostremos el *timeline* y comprobemos visualmente si realmente es o no planificable dicho conjunto de tareas.



Tema 6. Planificación de Tiempo Real

- Test de planificabilidad utilizando RMA

	Periodo (T)	T. Comput. (C)	Prioridad (P)	Utilización ($U = C/T$)
T1	50	12	1	0.24
T2	40	10	2	0.25
T3	30	10	3	0.33
Utilización Global de la CPU				0.82



Tema 6. Planificación de Tiempo Real

- Test de planificabilidad utilizando RMA
 - Mostremos otro ejemplo de utilización.

	Periodo (T)	T. Comput. (C)	Prioridad (P)	Utilización (U = C/T)
T1	80	32	1	0.400
T2	40	5	2	0.125
T3	16	4	3	0.250
Utilización Global de la CPU				0.775

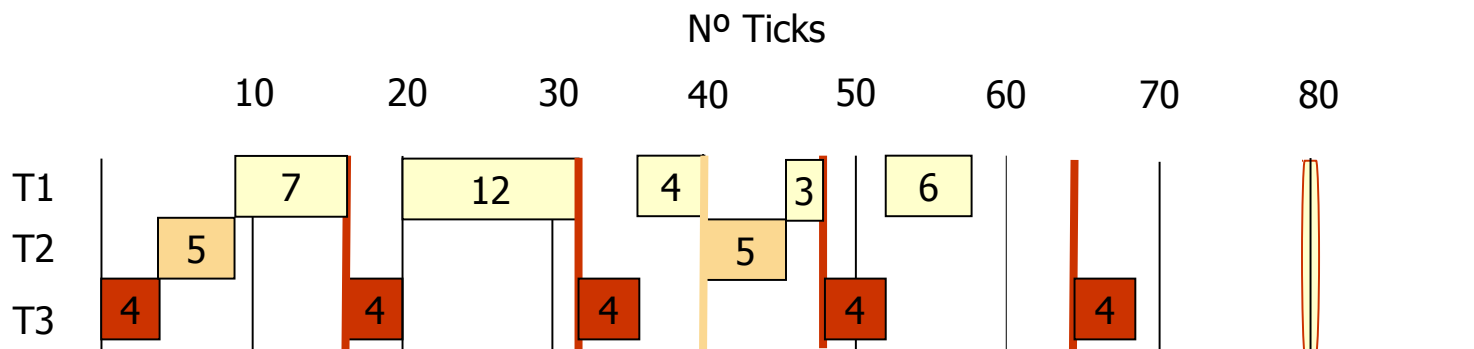
- La utilización de la CPU es de 0.775, mientras que $N(2^{1/N}-1)$ es igual a $3(2^{1/3}-1) = 0.78$; por lo que se cumple el test de planificabilidad.
- Mostremos el *timeline* y comprobemos visualmente si realmente es o no planificable dicho conjunto de tareas.



Tema 6. Planificación de Tiempo Real

• Test de planificabilidad utilizando RMA

	Periodo (T)	T. Comput. (C)	Prioridad (P)	Utilización ($U = C/T$)
T1	80	32	1	0.400
T2	40	5	2	0.125
T3	16	4	3	0.250
Utilización Global de la CPU				0.775



Tema 6. Planificación de Tiempo Real

- Test de planificabilidad utilizando RMA
 - Mostremos otro ejemplo de utilización.

	Periodo (T)	T. Comput. (C)	Prioridad (P)	Utilización (U = C/T)
T1	80	40	1	0.50
T2	40	10	2	0.25
T3	20	5	3	0.25
Utilización Global de la CPU				1.00

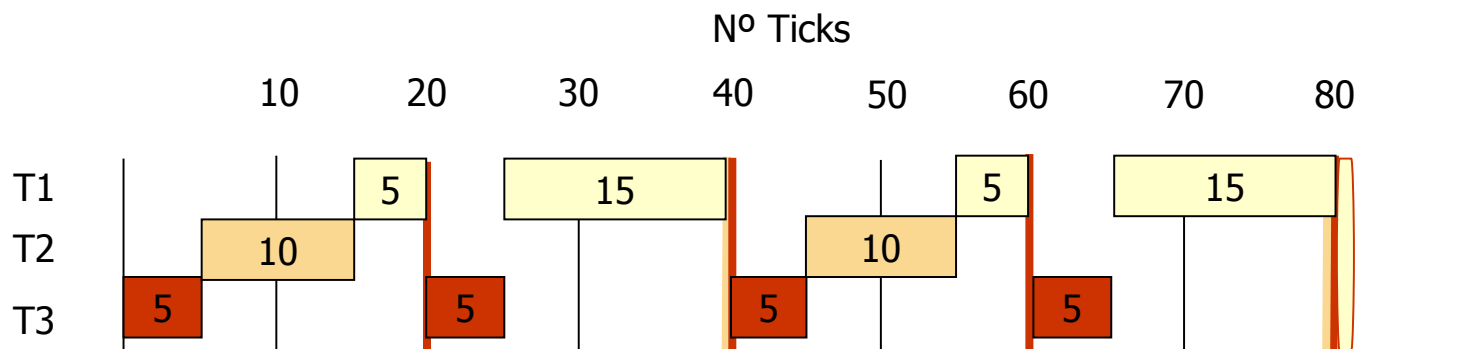
- La utilización de la CPU es del 100%, mientras que $N(2^{1/N}-1)$ es igual a $3(2^{1/3}-1) = 0.78$; por lo que se no se cumpliría el test de planificabilidad.
- Mostremos el *timeline* y comprobemos visualmente si realmente es o no planificable dicho conjunto de tareas.



Tema 6. Planificación de Tiempo Real

Test de planificabilidad utilizando RMA

	Periodo (T)	T. Comput. (C)	Prioridad (P)	Utilización ($U = C/T$)
T1	80	40	1	0.50
T2	40	10	2	0.25
T3	20	5	3	0.25
Utilización Global de la CPU				1.00



Tema 6. Planificación de Tiempo Real

• Análisis de tiempos de respuesta para RMA

- Existe otro test que permite obtener una condición suficiente y necesaria para comprobar si todas las tareas cumplen sus *deadlines*.
- Este test se basa en analizar el tiempo de respuesta real de cada tarea.
 - Para la tarea de máxima prioridad, el tiempo de respuesta del peor caso (R_i) es igual al tiempo de tiempo de computación (C_i); sin embargo, el resto de procesos de menor prioridad sufrirán **interferencias** de procesos de mayor prioridad. En general, un proceso genérico i : $R_i = C_i + I_i$, donde I_i es la máxima interferencia que un proceso sufrirá.
 - La máxima interferencia posible para un proceso con prioridad j se producirá cuando todas las tareas de mayor prioridad se lancen en el mismo instante que dicho proceso.
 - Se define el tiempo de respuesta del peor caso como:

$$R_i = C_i + \sum_{j \in mp(i)} \text{ceil}\left(\frac{R_i}{T_j}\right) \cdot C_j$$

- Donde $mp(i)$ es el conjunto de todas las tareas con mayor prioridad que el proceso i , T_j es el periodo del proceso j .
- La principal dificultad se encuentra en calcular el tiempo de respuesta en el peor caso de cada proceso en cada iteración (ver *Burns, 1997*).



Tema 6. Planificación de Tiempo Real

- Algoritmo de ordenación de prioridades por "Deadline" Monótono (DMPO)
 - El esquema RMA es óptimo cuando $D=T$ (el *deadline* de cada tarea coincide con el periodo de cada tarea). Leung y Whitehead (1982) mostraron que para $D<T$ se puede tener una formulación similar, denominada "Deadline Monotonic Priority Ordering (DMPO)".
 - La prioridad de un proceso es inversamente proporcional a su *deadline* ($D_i < D_j \Rightarrow P_i > P_j$).
 - El método DMPO es óptimo para cualquier conjunto de procesos. Si un conjunto de procesos es planificable por algún esquema de prioridades, entonces dicho conjunto de procesos es planificable utilizando DMPO.



Tema 6. Planificación de Tiempo Real

- **Asignación dinámica de Prioridad: EDF (Earliest Deadline First)**
 - El Algoritmo EDF es un algoritmo de asignación dinámica de prioridades. Las prioridades de las tareas van modificándose, proporcionándose mayor prioridad a las tareas que tienen más cercano su *deadline*.
 - Con EDF no es necesario realizar la suposición de que las tareas son periódicas. El resto de suposiciones realizadas para RMA se toman para EDF.
 - EDF es un algoritmo óptimo en uniprosesadores: si un conjunto de tareas no es viable con EDF, no existe ningún otro algoritmo de planificación que los haga viables.
 - El test de planificación para EDF cuando todas las tareas son periódicas y sus *deadlines* coinciden con sus periodos, es simple:
 - Si la utilización total del conjunto de tareas es menor que 1, el conjunto de tareas se puede planificar en un procesador usando EDF.
 - Sin embargo, si no se cumplen que las tareas sean periódicas o que los *deadlines* no coinciden con los periodos, no existe ningún mecanismo para comprobar la planificabilidad: hay que ejecutarlo y comprobarlo experimentalmente.



Tema 6. Planificación de Tiempo Real

1. Estrategias de planificación
2. Estructuración de prioridades
3. Asignación de prioridades
- ➔ 4. Interacción entre Procesos y Bloqueos
 1. Inversión de prioridad
 2. Herencia de prioridad
 3. Techo de prioridad original y Techo de prioridad inmediato



Tema 6. Planificación de Tiempo Real

• Inversión de prioridad

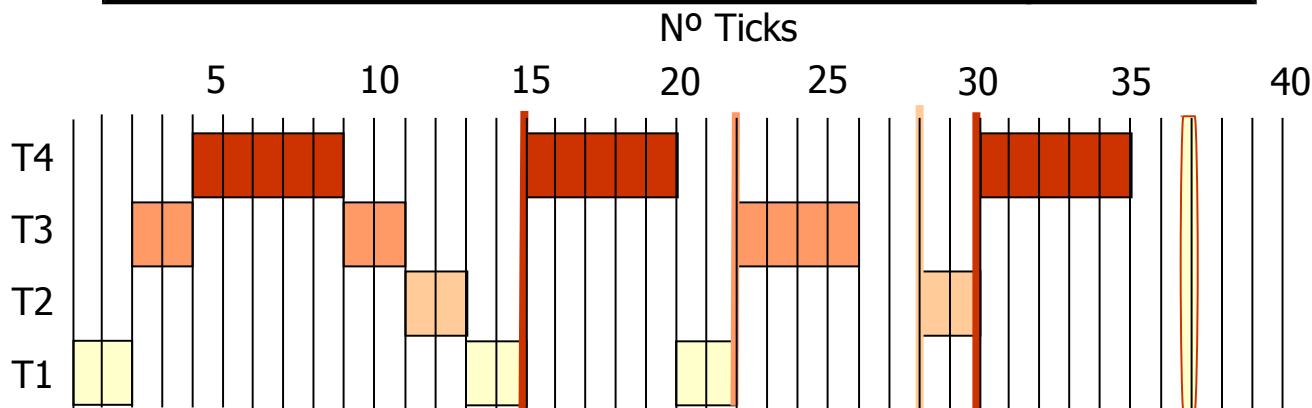
- Al introducir los supuestos para los algoritmos de planificación con prioridad se tomaba como supuesto que el cambio de contexto se podía realizar en cualquier instante: la realidad confirma que este supuesto es muy poco realista.
- Cuando un proceso de alta prioridad no puede ejecutarse porque intenta acceder a un recurso que está en posesión de otro proceso de menor prioridad se dice que se ha producido una **inversión de prioridad**.
- El problema de la inversión de prioridad puede provocar que una tarea de alta prioridad no llegue a cumplir su *deadline* por culpa de otras tareas de menor prioridad.

Tema 6. Planificación de Tiempo Real

• Inversión de prioridad

	Inicio	Periodo (T)	T.Comput. (C)	Prioridad (P)	Utilización ($U = C/T$)
T1	0	36	6	1	0.167
T2	2	28	2	2	0.071
T3	2	22	4	3	0.182
T4	4	15	5	4	0.333
Utilización Global de la CPU					0.753

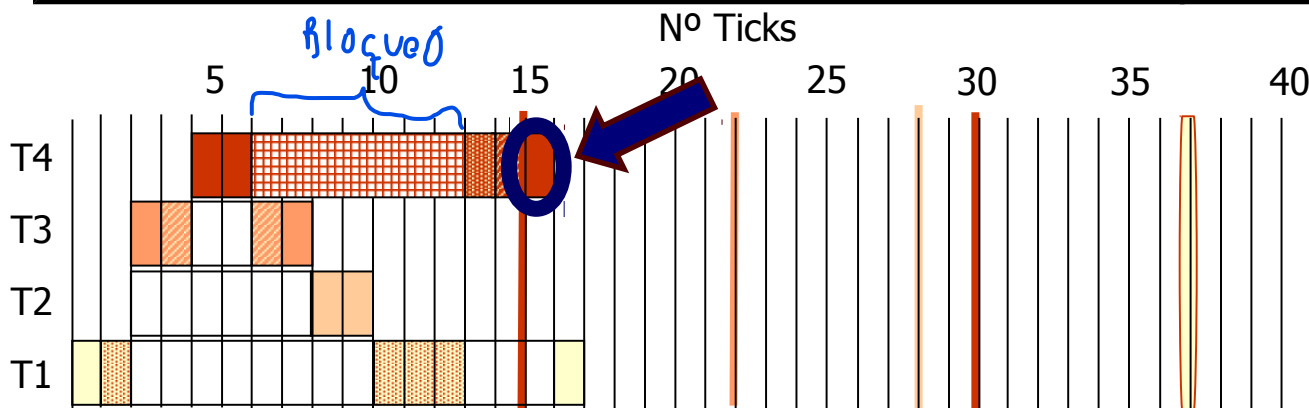
Planificable RMA



Tema 6. Planificación de Tiempo Real

• Inversión de prioridad

	Inicio	Periodo (T)	T.Comput. (C)	Prioridad (P)	Secuencia Ejecución	Utilización ($U = C/T$)
T1	0	36	6	1	EQQQQE	0.167
T2	2	28	2	2	EE	0.071
T3	2	22	4	3	EVVE	0.182
T4	4	15	5	4	EEQVE	0.333
Utilización Global de la CPU						0.753



Tema 6. Planificación de Tiempo Real

• Inversión de prioridad

- En el ejemplo anterior, T4 ha perdido su *deadline*, porque ha permanecido bloqueada demasiado tiempo: no solo ha sido bloqueado por T1, sino también por T2 y T3. Es decir, que la tarea de máxima prioridad ha sido adelantada por todas las tareas de menor prioridad.
- El bloqueo de T1 a T4 debido al intento de T4 de acceder al recurso Q es inevitable, ya que está en posesión previa por T1.
- Sin embargo, el adelanto de T2 y T3 no solo es improductivo sino que afecta gravemente a la planificación.



Tema 6. Planificación de Tiempo Real

• Herencia de prioridad

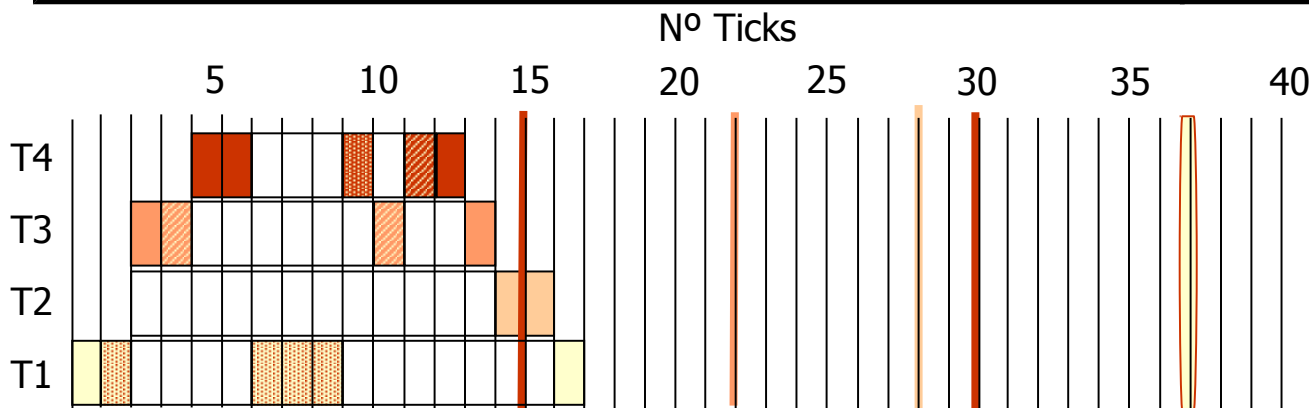
- Para mitigar el efecto de la inversión de prioridad en los casos inevitables y evitar que otras tareas de menor prioridad afecten a otros de mayor prioridad se ha propuesto el método de **herencia de prioridad**.
- Con el método de la herencia de prioridad, la tarea de menor prioridad que está en posesión del recurso hereda temporalmente la prioridad de la tarea de mayor prioridad que desea acceder al recurso, hasta que el de menor prioridad finalice de utilizar dicho recurso.
- De esta manera, la tarea de menor prioridad no es interrumpida por otras tareas con prioridad inferior que el de mayor prioridad.
- La **herencia de prioridad implica tener un mecanismo dinámico de prioridad**.
- Un método sencillo de herencia de prioridad sería:
 - la prioridad de un proceso es el máximo entre su propia prioridad inicial y las prioridades de todos los demás procesos que están bloqueados por él.



Tema 6. Planificación de Tiempo Real

Herencia de prioridad

	Inicio	Periodo (T)	T.Comput. (C)	Prioridad (P)	Secuencia Ejecución	Utilización ($U = C/T$)
T1	0	36	6	1	EQQQQE	0.167
T2	2	28	2	2	EE	0.071
T3	2	22	4	3	EVVE	0.182
T4	4	15	5	4	EEQVE	0.333
Utilización Global de la CPU						0.753



Tema 6. Planificación de Tiempo Real

• Techo de Prioridad Original (OCP)

- El método OCP (*Original Ceiling Priority Protocol*) es un método complejo para limitar el número máximo de bloqueos que puede sufrir una tarea de alta prioridad.
- Con OCP:
 - Todo proceso de alta prioridad puede ser bloqueado como máximo una única vez durante su ejecución por procesos de menor prioridad.
 - Se previenen los *deadlocks*.
 - Se previenen los bloqueos transitivos.
 - Se garantiza el acceso en exclusión mutua a los recursos (por el propio protocolo).
- Los protocolos de techo prioridad se describen mejor en términos de recursos protegidos.



Tema 6. Planificación de Tiempo Real

• Techo de Prioridad Original (OCP)

• El algoritmo OCP tiene la siguiente forma:

- Cada proceso tiene una prioridad estática previamente asignada (por ejemplo utilizando RMA).
- Cada recurso tiene un valor de techo máximo de prioridad, que es el valor de prioridad del proceso de mayor prioridad que lo utiliza.
- Cada proceso tiene una prioridad dinámica que es el máximo entre su propia prioridad estática y las prioridades que herede debido al bloqueo de procesos de mayor prioridad.
- Un proceso podrá adquirir un recurso solo si su prioridad dinámica es mayor que el techo de cualquier recurso adquirido por todas las demás tareas.



Tema 6. Planificación de Tiempo Real

• Techo de Prioridad Original (OCPP)

• En esencia el algoritmo OCPP:

- Asegura que si un recurso ha sido adquirido por un proceso P_1 y pudiera provocar el bloqueo de otro proceso de más prioridad P_2 , entonces ningún otro recurso que pudiese bloquear P_2 puede ser adquirido por otro proceso que no sea P_1 . De esta manera, proporciona a P_1 todos los recursos necesarios para que se ejecute lo más rápido posible.
- Un proceso podrá ser retardado no sólo por intentar adquirir un recurso que haya sido adquirido previamente, sino cuando la adquisición de un recurso libre pudiera provocar en un futuro un *deadlock* o el bloqueo de procesos de mayor prioridad.
- La adquisición de un primer recurso en el sistema está permitido. El protocolo pretende asegurarse que un segundo recurso puede ser adquirido solamente si no existe un proceso de mayor prioridad que utilice ambos.

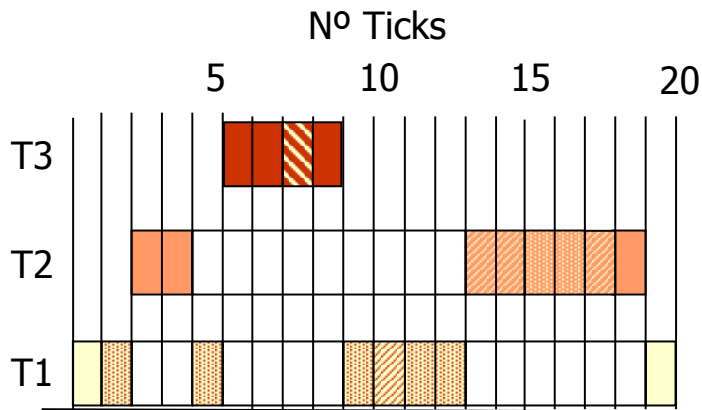


Tema 6. Planificación de Tiempo Real

OCPP

- Supongamos 3 Tareas: T1,T2,T3 y 3 regiones críticas Q,V,X. La prioridad de $P(T3)=3$; $P(T2)=2$; $P(T1)=1$. Inicio(T1)=0; Inicio(T2)=2; Inicio(T3)=5.
- La estructura de ejecución sería $T1=\{EXXXVXXEE\}$, $T2=\{EEVVXXVE\}$, $T3=\{EEQE\}$. Donde E es un tick de ejecución; Q, V y X representan respectivamente un tick de ejecución con el recurso Q, V o X en exclusiva.
- Los recursos se obtienen desde el primer tick que aparecen hasta el último tick, obteniendo otros recursos sin liberar los anteriores.

Recurso	Q	V	X
Techo	3	2	2



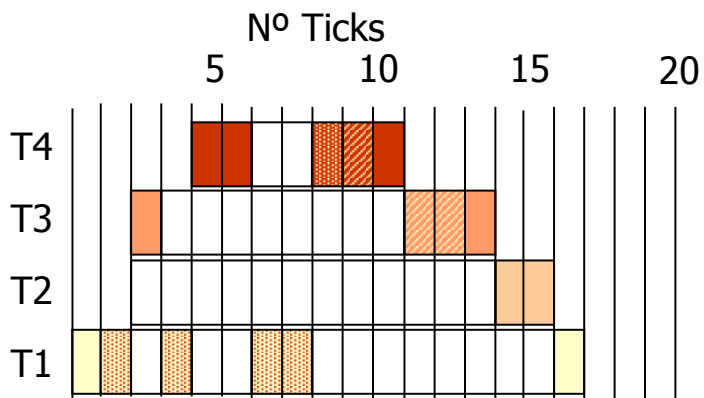
- Comienza T1. Adquiere X.
- Comienza T2. Se suspende T1.
- T2 intenta adquirir V, sin embargo, la prioridad de T2 no es superior al techo de prioridad de X (recurso bloqueado por T1): T2 se bloquea fuera de Sección Crítica. T1 hereda la prioridad de T2.
- T1 continúa en ejecución con X.
- Comienza T3. Adquiere Q, ya que la prioridad de $T3=3 >$ techo de prioridad de $X = 2$. Finaliza T3.
- T1 continúa en ejecución con X. Adquiere V, prioridad dinámica de $T1 = 2$ y el resto de tareas no tiene ningún recurso protegido. Libera V. Continúa su ejecución con X. Libera X. Reduce su prioridad a la estática original.
- T2 se despierta. Adquiere V. Adquiere X. Libera X. Libera V. Finaliza la ejecución.
- T1 finaliza la ejecución.

Tema 6. Planificación de Tiempo Real

OCPP

	Inicio	Prioridad (P)	Secuencia Ejecución
T1	0	1	EQQQQE
T2	2	2	EE
T3	2	3	EVVE
T4	4	4	EEQVE

Recurso	Q	V
Techo	4	4



- Comienza T1. Adquiere Q.
- Comienza T2 y T3. Se suspenden T1 y T2.
- T3 intenta adquirir V, sin embargo, la prioridad de T3 no es superior al techo de prioridad de Q (recurso bloqueado por T1): T3 se bloquea fuera de Sección Crítica. Los techos de prioridad se examinan solo cuando los procesos solicitan recursos. T1 hereda la prioridad de T3.
- T1 continúa en ejecución con Q.
- Comienza T4. Intenta adquirir Q, se bloquea porque está en posesión de T1. T1 hereda la prioridad de T4.
- T1 continúa en ejecución con Q. Libera Q. Reduce su prioridad a la estática original.
- T4 se despierta y como no hay ningún recurso bloqueado, adquiere Q. Adquiere V. Libera ambos y finaliza.
- T3 se despierta. Como no hay recursos bloqueados, adquiere V. Libera V. Finaliza la ejecución.
- T2 se despierta. Ejecuta sin solicitar ningún recurso.
- T1 se despierta. Finaliza la ejecución.

Tema 6. Planificación de Tiempo Real

• Techo de Prioridad Inmediata (ICPP)

- El método ICPP (*Immediate Ceiling Priority Protocol*) es un método algo más simple que el OCPP para limitar el número máximo de bloqueos que puede sufrir una tarea de alta prioridad.
- Con ICPP:
 - Aumenta la prioridad de una tarea mediante herencia de prioridad incrementando inmediatamente en cuanto se adquiere el recurso (frente a OCPP que lo aumentaba únicamente cuando estaba bloqueando a un proceso de mayor prioridad).
- ICPP es algo más fácil de implementar que OCPP, ya que no tiene que monitorizar todas las relaciones entre procesos cuando se produce un bloqueo y tiene resultados similares en cuanto al tiempo del peor caso.
- ICPP produce menos cambios de contexto que OCPP.
- ICPP produce más cambios de prioridad que OCPP.



Tema 6. Planificación de Tiempo Real

• Techo de Prioridad Inmediata (ICPP)

- El algoritmo ICPP tiene la siguiente forma:
 - Cada proceso tiene una prioridad estática previamente asignada (por ejemplo utilizando RMA).
 - Cada recurso tiene un valor de techo máximo de prioridad, que es el valor de prioridad del proceso de mayor prioridad que lo utiliza.
 - Cada proceso tiene una prioridad dinámica que es el máximo entre su propia prioridad estática y los valores de techo de prioridad de cualquier recurso que adquiera.
- Un proceso sufrirá bloqueos solo al principio de su ejecución. Una vez que comienza todos los recursos que necesite deberán estar libres:
 - Si no lo están es porque algún proceso tendrá una prioridad (estática o dinámica) igual o superior a él y tendrá que posponer su ejecución hasta que éste libere los recursos.

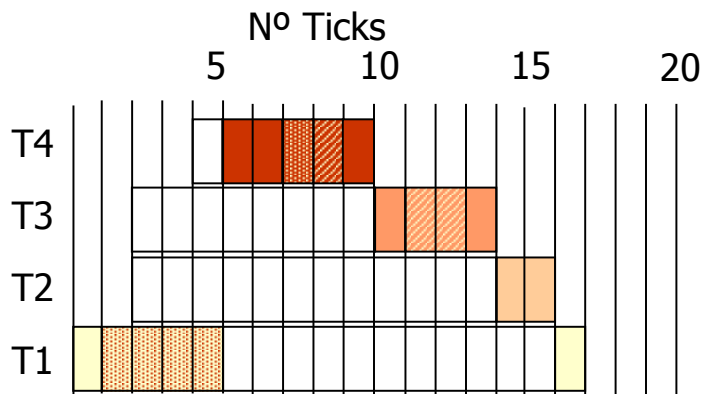


Tema 6. Planificación de Tiempo Real

ICPP

	Inicio	Prioridad (P)	Secuencia Ejecución
T1	0	1	EQQQQE
T2	2	2	EE
T3	2	3	EVVE
T4	4	4	EEQVE

Recurso	Q	V
Techo	4	4



- Comienza T1. Como no hay recursos bloqueados, adquiere Q. Aumenta su prioridad al techo de prioridad de Q (4).
- Comienza T2 y T3. Se suspenden T2 y T3. Sigue ejecutándose T1 sin ser replanificado.
- Comienza T4. Se suspende T4. Sigue ejecutándose T1 sin ser replanificado.
- T1 continúa en ejecución con Q. Libera Q. Reduce su prioridad a la estática original.
- Comienza T4. Como no hay recursos bloqueados, adquiere Q. Adquiere V. Libera Q. Libera V. Finaliza su ejecución
- T3 se despierta. Como no hay recursos bloqueados, adquiere V. Libera V. Finaliza la ejecución.
- T2 se despierta. Ejecuta sin solicitar ningún recurso.
- T1 se despierta. Finaliza la ejecución.

Preguntas ...

Volver

